

CSE101 Test 4 Practice – All Problems with Two Pseudocode Solutions

Easy

- [1. Two Sum](#) - Hash Map / Two Pointers
- [136. Single Number](#) - Bit Manipulation
- [496. Next Greater Element I](#) - Stack
- [283. Move Zeroes](#) - Two Pointers
- [169. Majority Element](#) - Greedy
- [268. Missing Number](#) - Bit Manipulation
- [1213. Intersection of Three Sorted Arrays](#) - Three Pointers
- [35. Search Insert Position](#) - Binary Search
- [141. Linked List Cycle](#) - Two Pointers
- [121. Best Time to Buy and Sell Stock](#) - Greedy
- [1331. Rank Transform of an Array](#) - Array Index Mapping
- [1064. Fixed Point](#) - Binary Search
- [278. First Bad Version](#) - Binary Search
- [155. Min Stack](#) - Stack
- [232. Implement Queue using Stacks](#) - Stack
- [20. Valid Parentheses](#) - Stack
- [703. Kth Largest Element in a Stream](#) - Heap

Medium

- [167. Two Sum II - Input Array Is Sorted](#) - Two Pointers
- [378. Kth Smallest Element in a Sorted Matrix](#) - Heap / Binary Search
- [53. Maximum Subarray](#) - Dynamic Programming
- [503. Next Greater Element II](#) - Stack
- [2161. Partition Array According to Given Pivot](#) - Two Pointers
- [15. 3Sum](#) - Sorting + Two Pointers
- [56. Merge Intervals](#) - Sorting
- [Index of an Extra Element](#) - Binary Search
- [162. Find Peak Element](#) - Binary Search
- [1901. Find a Peak Element II](#) - Divide & Conquer
- [142. Linked List Cycle II](#) - Two Pointers
- [122. Best Time to Buy and Sell Stock II](#) - Greedy
- [148. Sort List](#) - Divide & Conquer
- [240. Search a 2D Matrix II](#) - Two Pointers
- [1008. Construct Binary Search Tree from Preorder Traversal](#) - Recursion
- [2080. Range Frequency Queries](#) - Binary Search
- [380. Insert Delete GetRandom O\(1\)](#) - Hash Map + Array
- [235. Lowest Common Ancestor of a Binary Search Tree](#) - Binary Search (Tree Traversal)
- [347. Top K Frequent Elements](#) - Heap
- [98. Validate Binary Search Tree](#) - DFS (min/max) / Inorder
- [692. Top K Frequent Words](#) - Heap
- [215. Kth Largest Element in an Array](#) - Heap / Quickselect
- [Convert Min Heap to Max Heap](#) - Heapify / Build-Heap
- [701. Insert into a Binary Search Tree](#) - Recursion
- [450. Delete Node in a BST](#) - Recursion
- [669. Trim a Binary Search Tree](#) - Recursion

Hard

- [4. Median of Two Sorted Arrays](#) - Binary Search
- [K-th Element of Merged Two Sorted Arrays](#) - Binary Search
- [23. Merge k Sorted Lists](#) - Heap / Divide & Conquer
- [295. Find Median from Data Stream](#) - Heap
- [632. Smallest Range Covering Elements from K Lists](#) - Heap

Array Questions

Question 1: Two numbers sum to x (or closest sum)

Given a number x, find if two numbers add up to x (or the closest sum to x)

1. Two Sum: <https://leetcode.com/problems/two-sum/>

<https://youtu.be/KLIXCFG5TnA>

167. Two Sum II - Input Array Is Sorted: <https://neetcode.io/problems/two-integer-sum-ii/question>

<https://www.youtube.com/watch?v=cQ1Oz4ckceM>

Solution 1 (Brute Force)

```
TwoSumExists_Brute(A, x):
    for i in 0..len(A)-1:
        for j in i+1..len(A)-1:
            if A[i] + A[j] == x:
                return true
    return false
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
TwoSumExists(A, x):
    sort(A)
    l = 0; r = len(A)-1
    while l < r:
        s = A[l] + A[r]
        if s == x: return true
        if s < x: l += 1
        else: r -= 1
    return false

TwoSumClosestPair(A, x):
    sort(A)
    l = 0; r = len(A)-1
    bestPair = (A[l], A[r])
    bestDiff = abs((A[l]+A[r]) - x)
    while l < r:
        s = A[l] + A[r]
        if abs(s - x) < bestDiff:
            bestDiff = abs(s - x)
            bestPair = (A[l], A[r])
        if s < x: l += 1
        else if s > x: r -= 1
        else: break
    return bestPair
```

Total time: $O(n \log n)$

Extra space: $O(1)$ (ignoring sort recursion)

Array Questions

Question 2: Median of union of two sorted arrays

Find the median of the union of 2 sorted arrays

4. Median of Two Sorted Arrays: <https://leetcode.com/problems/median-of-two-sorted-arrays>

<https://www.youtube.com/watch?v=xMBwzNvXmms>

Solution 1 (Brute Force)

```
MedianTwoSorted_Merge(A, B):
    i = 0; j = 0
    M = empty list
    while i < len(A) and j < len(B):
        if A[i] <= B[j]:
            append(M, A[i]); i += 1
        else:
            append(M, B[j]); j += 1
    while i < len(A): append(M, A[i]); i += 1
    while j < len(B): append(M, B[j]); j += 1

    n = len(M)
    if n % 2 == 1: return M[n//2]
    return (M[n//2 - 1] + M[n//2]) / 2
```

Total time: $O(n+m)$

Extra space: $O(n+m)$

Solution 2 (Improved)

```
MedianTwoSorted(A, B):
    if len(A) > len(B): swap(A, B)
    n = len(A); m = len(B)
    total = n + m
    half = (total + 1) // 2

    lo = 0; hi = n
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i

        Aleft = -INF if i == 0 else A[i-1]
        Aright = +INF if i == n else A[i]
        Bleft = -INF if j == 0 else B[j-1]
        Bright = +INF if j == m else B[j]

        if Aleft <= Bright and Bleft <= Aright:
            if total % 2 == 1:
                return max(Aleft, Bleft)
            else:
                return (max(Aleft, Bleft) + min(Aright, Bright)) / 2
        else if Aleft > Bright:
            hi = i - 1
        else:
            lo = i + 1
```

Total time: $O(\log \min(n,m))$

Extra space: $O(1)$

Array Questions

Question 3: *kth smallest in union of two sorted arrays*

Find the kth smallest element in the union of two sorted arrays

K-th Element of Merged Two Sorted Arrays : <https://www.geeksforgeeks.org/k-th-element-two-sorted-arrays/>
<https://www.youtube.com/watch?v=ZDllyWxyYg8>

Solution 1 (Brute Force)

```
KthTwoSorted_MergeUntil(A, B, k):
    i = 0; j = 0
    count = 0
    while i < len(A) or j < len(B):
        if j == len(B) or (i < len(A) and A[i] <= B[j]):
            val = A[i]; i += 1
        else:
            val = B[j]; j += 1
        count += 1
        if count == k: return val
```

Total time: $O(k)$ **or worst** $O(n+m)$

Extra space: $O(1)$

Solution 2 (Improved)

```
KthTwoSorted(A, B, k):
    if len(A) > len(B): swap(A, B)
    n = len(A); m = len(B)

    lo = max(0, k - m)
    hi = min(k, n)

    while lo <= hi:
        i = (lo + hi) // 2
        j = k - i

        Aleft = -INF if i == 0 else A[i-1]
        Aright = +INF if i == n else A[i]
        Bleft = -INF if j == 0 else B[j-1]
        Bright = +INF if j == m else B[j]

        if Aleft <= Bright and Bleft <= Aright:
            return max(Aleft, Bleft)
        else if Aleft > Bright:
            hi = i - 1
        else:
            lo = i + 1
```

Total time: $O(\log \min(n,m))$

Extra space: $O(1)$

Array Questions

Question 4: Median/kth for L sorted arrays

Above 2 questions for L sorted arrays

378. Kth Smallest Element in a Sorted Matrix: <https://leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix>
<https://www.youtube.com/watch?v=gQuH27Xz5mk>

Solution 1 (Brute Force)

```
FlattenAndSortMedian(Arrays):  
    M = empty list  
    for each arr in Arrays:  
        for each x in arr:  
            append(M, x)  
    sort(M)  
    n = len(M)  
    if n % 2 == 1: return M[n//2]  
    return (M[n//2 - 1] + M[n//2]) / 2
```

Total time: $O(N \log N)$

Extra space: $O(N)$

Solution 2 (Improved)

```
KthLSorted(Arrays, k):  
    heap = empty min-heap of (value, arrayId, index)  
    for t in 0..L-1:  
        if len(Arrays[t]) > 0:  
            push(heap, (Arrays[t][0], t, 0))  
  
    count = 0  
    while heap not empty:  
        (v, t, i) = popMin(heap)  
        count += 1  
        if count == k: return v  
        if i + 1 < len(Arrays[t]):  
            push(heap, (Arrays[t][i+1], t, i+1))  
  
MedianLSorted(Arrays):  
    N = sum of lengths  
    if N % 2 == 1:  
        return KthLSorted(Arrays, (N+1)//2)  
    a = KthLSorted(Arrays, N//2)  
    b = KthLSorted(Arrays, N//2 + 1)  
    return (a + b) / 2
```

Total time: Kth: $O(k \log L)$; **Median worst** $O(N \log L)$

Extra space: $O(L)$ heap

Array Questions

Question 5: Merge k sorted arrays

We discussed in class how to merge two sorted arrays. What about merging k sorted arrays?

23. Merge k Sorted Lists: <https://leetcode.com/problems/merge-k-sorted-lists>

<https://www.youtube.com/watch?v=2J2c3oGKBI4>

<https://www.youtube.com/watch?v=q5a5OiGbT6Q>

Solution 1 (Brute Force)

```
MergeKLists(lists):
    if lists is empty: return null
    while number of lists > 1:
        temp = []
        for i = 0 to len(lists)-1 step 2:
            l1 = lists[i]
            l2 = lists[i+1] if exists else null
            temp.append( MergeTwoLists(l1, l2) )
        lists = temp
    return lists[0]
```

Total time: $O(\text{totalNodes} \cdot L)$ in the worst case (unbalanced / sequential accumulation)

Extra space: $O(1)$ auxiliary (if reusing nodes) + output (already exists)

Solution 2 (Improved)

```
MergeKLists(lists):
    if lists is empty: return null
    while number of lists > 1:
        temp = []
        for i = 0 to len(lists)-1 step 2:
            l1 = lists[i]
            l2 = lists[i+1] if exists else null
            temp.append( MergeTwoLists(l1, l2) )
        lists = temp
    return lists[0]
```

```
MergeTwoLists(l1, l2):
    dummy = new node
    tail = dummy
    while l1 != null and l2 != null:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 != null else l2
    return dummy.next
```

Total time: $O(\text{totalNodes} \log L)$

Extra space: $O(L)$ for temp

Array Questions

Question 6: All numbers appear twice except one

You have an unsorted array of integers where every integer appears exactly twice, except one integer that appears once. Idea: Create a hash table for frequency for each item in array. Looping through array, add item to hash table if it doesn't exist, else increment item value to 2

136. Single Number: <https://leetcode.com/problems/single-number/>

<https://www.youtube.com/watch?v=qMPX1AOa83k>

Solution 1 (Brute Force)

```
UniqueOnce_Brute(A):
    for i in 0..len(A)-1:
        count = 0
        for j in 0..len(A)-1:
            if A[j] == A[i]:
                count += 1
        if count == 1: return A[i]
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
UniqueOnce(A):
    ans = 0
    for x in A:
        ans = ans XOR x
    return ans
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 7: Maximum-sum contiguous subarray

Given an array, describe an algo to identify the contiguous subarray with the maximum sum. For example, if the input is [1, -3, 5, -2, 9, -6, 4], the output should be [5, -2, 9]

53. Maximum Subarray: <https://leetcode.com/problems/maximum-subarray>

<https://neetcode.io/problems/maximum-subarray>

<https://www.youtube.com/watch?v=5WZI3MMT0Eg>

Solution 1 (Brute Force)

```
MaxSubarray_Brute(A):
    best = -INF
    for i in 0..len(A)-1:
        s = 0
        for j in i..len(A)-1:
            s += A[j]
            if s > best:
                best = s
                bestL = i; bestR = j
    return (best, bestL, bestR)
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MaxSubarray_Kadane(A):
    cur = best = A[0]

    for i = 1 to len(A)-1:
        cur = max(A[i], cur + A[i])
        best = max(best, cur)
    return best
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 8: For each i , nearest index j with $A[j] > A[i]$

Given an array A of numbers, for every index i , find the nearest index j such that $A[j] > A[i]$. If none exist, report -1 . Output these indices sorted by the corresponding i .

503. Next Greater Element II: <https://leetcode.com/problems/next-greater-element-ii> or

496. Next Greater Element I: <https://leetcode.com/problems/next-greater-element-i>

<https://www.youtube.com/watch?v=5rRJw5hFVks> or <https://www.youtube.com/watch?v=W44bZssIWcg>

Solution 1 (Brute Force)

```
NearestGreater_Brute(A):
    n = len(A)
    ans = array size n filled -1
    for i in 0..n-1:
        bestDist = +INF
        bestJ = -1
        for j in 0..n-1:
            if j != i and A[j] > A[i]:
                d = abs(j - i)
                if d < bestDist:
                    bestDist = d
                    bestJ = j
        ans[i] = bestJ
    return ans
```

Total time: $O(n^2)$

Extra space: $O(n)$ output

Solution 2 (Improved)

```
NearestGreater(A):
    n = len(A)
    left = array size n filled -1
    right = array size n filled -1

    stack = empty stack # indices, strictly decreasing values
    for i in 0..n-1:
        while stack not empty and A[stack.top] <= A[i]:
            stack.pop()
        left[i] = stack.top if stack not empty else -1
        stack.push(i)

    stack = empty stack
    for i in n-1 down to 0:
        while stack not empty and A[stack.top] <= A[i]:
            stack.pop()
        right[i] = stack.top if stack not empty else -1
        stack.push(i)

    ans = array size n filled -1
    for i in 0..n-1:
        L = left[i]; R = right[i]
        if L == -1 and R == -1: ans[i] = -1
        else if L == -1: ans[i] = R
        else if R == -1: ans[i] = L
        else:
            if (i - L) <= (R - i): ans[i] = L
            else: ans[i] = R
    return ans
```

Total time: $O(n)$

Extra space: $O(n)$ (stack + output)

Array Questions

Question 9: Move all zeros to the right end (order of others doesn't matter)

Given an array of integers, move all the zeros to the right end. The order of the others doesn't matter. Use $O(1)$ extra storage../

283. Move Zeroes: <https://leetcode.com/problems/move-zeroes/>

<https://www.youtube.com/watch?v=aayNRwUN3Do>

Solution 1 (Brute Force)

```
MoveZeros_Brute(A):
    # repeatedly find a zero and swap it rightward one step at a time
    n = len(A)
    changed = True
    while changed:
        changed = False
        for i in 0..n-2:
            if A[i] == 0 and A[i+1] != 0:
                swap(A[i], A[i+1])
                changed = True
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MoveZeros(A):
    left = 0
    for right = 0 to len(A) - 1:
        if A[right] != 0:
            swap(A[right], A[left])
            left = left + 1
    return A
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 10: Stable reorder negatives, then zeros, then positives ($O(1)$ extra)

Given an array of integers, reorder so that all the negative integers appear first, then the zeroes, and finally the positive integers. If two integers have the same sign, their order should not be changed. Use $O(1)$ extra storage.

2161. Partition Array According to Given Pivot: <https://leetcode.com/problems/partition-array-according-to-given-pivot>

<https://www.youtube.com/watch?v=OX0bCSG7EfE>

```
Solution 1 (Brute Force)
    StableReorderBySign(A):
        n = length(A)
        write = 0

        # 1) Move negatives to front (stable)
        for i from 0 to n-1:
            if A[i] < 0:
                temp = A[i]
                j = i
                while j > write:
                    A[j] = A[j-1]
                    j = j - 1
                A[write] = temp
                write = write + 1

        write2 = write

        # 2) Move zeros after negatives (stable)
        for i from write to n-1:
            if A[i] == 0:
                temp = A[i]
                j = i
                while j > write2:
                    A[j] = A[j-1]
                    j = j - 1
                A[write2] = temp
                write2 = write2 + 1

        return A
```

Total time: $O(n^2)$ worst case

Extra space: $O(1)$

```
Solution 2 (Faster time, but violates  $O(1)$  space constraint)
    Create three lists: neg, zero, pos
    Loop once through A:
        append to appropriate list
    Overwrite A with neg + zero + pos
```

Total time: $O(n)$

Extra space: $O(n)$

Array Questions

Question 11: Stream median

You have a stream of numbers. Implement an algorithm to return the median of the numbers seen so far.

295. Find Median from Data Stream: <https://leetcode.com/problems/find-median-from-data-stream>

<https://www.youtube.com/watch?v=itmhHWaHupI>

Solution 1 (Brute Force)

```
StreamMedian_Brute():
    A = empty list

    insert(x):
        append(A, x)
        sort(A)

    getMedian():
        n = len(A)
        if n % 2 == 1:
            return A[n//2]
        else:
            return (A[n//2 - 1] + A[n//2]) / 2
```

Total time: $O(n \log n)$, median $O(1)$

Extra space: $O(n)$

Solution 2 (Improved)

```
StreamMedian():
    maxHeap = empty max-heap    # lower half
    minHeap = empty min-heap    # upper half

    insert(x):
        if maxHeap.empty() or x <= maxHeap.top():
            maxHeap.push(x)
        else:
            minHeap.push(x)

        if maxHeap.size > minHeap.size + 1:
            minHeap.push(maxHeap.popTop())
        if minHeap.size > maxHeap.size:
            maxHeap.push(minHeap.popTop())

    getMedian():
        if maxHeap.size == minHeap.size:
            return (maxHeap.top() + minHeap.top()) / 2
        return maxHeap.top()
```

Total time: insert: $O(\log n)$, median $O(1)$

Extra space: $O(n)$

Array Questions

Question 12: Majority element ($> n/2$) and elements $> n/3$ with $O(1)$ extra storage

A majority element in an array is an element that appears more than $n/2$ times (where n is the size of the array). Give an algorithm to find a majority element. More generally, give an algorithm to find all elements that appear more than $n/3$ times. Use $O(1)$ extra storage.

169. Majority Element: <https://leetcode.com/problems/majority-element>

<https://www.youtube.com/watch?v=7pnhv842keE>

Solution 1 (Brute Force)

```
Majority_Brute(A):
    for i in 0..len(A)-1:
        count = 0
        for j in 0..len(A)-1:
            if A[j] == A[i]: count += 1
        if count > len(A)/2: return A[i]
    return NONE
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MajorityOverHalf(A):
    cand = NONE
    count = 0

    # Phase 1: Find candidate
    for each x in A:
        if count == 0:
            cand = x
            count = 1
        else if x == cand:
            count += 1
        else:
            count -= 1

    # Phase 2: Verify candidate
    occurrences = 0
    for each x in A:
        if x == cand:
            occurrences += 1

    if occurrences > len(A) // 2:
        return cand
    else:
        return NONE
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 13: Missing element(s) from 1..n using O(1) memory

Given an array of integers containing all the elements from 1,2,...,n except one of them. Find which one.

Try doing it using O(1) memory. What if two elements were missing?

268. Missing Number: <https://leetcode.com/problems/missing-number>

<https://www.youtube.com/watch?v=WnPLSRLSANE>

Solution 1 (Brute Force)

```
MissingOne_Brute(A, n):
    for v in 1..n:
        found = false
        for x in A:
            if x == v: found = true
        if not found: return v
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MissingOne(A, n):
    expected = n*(n+1)/2
    actual = 0
    for x in A: actual += x
    return expected - actual

MissingTwo(A, n):
    # uses sum and sum of squares
    S = n*(n+1)/2
    SQ = n*(n+1)*(2*n+1)/6
    s = 0
    sq = 0
    for x in A:
        s += x
        sq += x*x

    a_plus_b = S - s
    a2_plus_b2 = SQ - sq
    ab2 = (a_plus_b*a_plus_b - a2_plus_b2) / 2    # equals a*b

    # solve t^2 - (a_plus_b)t + ab2 = 0
    disc = a_plus_b*a_plus_b - 4*ab2
    a = (a_plus_b + sqrt(disc)) / 2
    b = a_plus_b - a
    return (a, b)
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 14: Detect a Pythagorean triple

Given an array of integers, determine if it contains a Pythagorean triple (integers a, b, c such that $c^2 = a^2 + b^2$).

15. 3Sum: <https://leetcode.com/problems/3sum>

<https://www.youtube.com/watch?v=jzZsG8n2R9A>

Solution 1 (Brute Force)

```
HasPythagoreanTriple_Brute(A):
    n = len(A)
    for i in 0..n-1:
        for j in i+1..n-1:
            for k in j+1..n-1:
                a = A[i]; b = A[j]; c = A[k]
                if a*a + b*b == c*c: return true
                if a*a + c*c == b*b: return true
                if b*b + c*c == a*a: return true
    return false
```

Total time: $O(n^3)$

Extra space: $O(1)$

Solution 2 (Improved)

```
HasPythagoreanTriple(A):
    B = empty list
    for x in A: append(B, x*x)
    sort(B)

    n = len(B)
    for c in n-1 down to 0:
        l = 0; r = c-1
        while l < r:
            s = B[l] + B[r]
            if s == B[c]: return true
            if s < B[c]: l += 1
            else: r -= 1
    return false
```

Total time: $O(n^2)$

Extra space: $O(n)$ (squared array)

Array Questions

Question 15: Merge intersecting ranges until disjoint

The input is a sequence of ranges $[x_1, y_1], [x_2, y_2], \dots$ (where each $x_i \leq y_i$). Given two ranges that intersect, merge them into a single range by taking the union of ranges. Keep doing this until you finally get disjoint ranges. Design an algorithm that determines this output.

56. Merge Intervals: <https://leetcode.com/problems/merge-intervals>

<https://www.youtube.com/watch?v=44H3cEC2fFM>

Solution 1 (Brute Force)

```
MergeRanges_Brute(R):
    changed = true
    while changed:
        changed = false
        for i in 0..len(R)-1:
            for j in i+1..len(R)-1:
                if R[i] intersects R[j]:
                    R[i] = union(R[i], R[j])
                    delete R[j]
                    changed = true
                    break
        if changed: break
    return R
```

Total time: $O(k^2)$ merges (can be worse with deletes)

Extra space: $O(1)$ extra (in-place list edits)

Solution 2 (Improved)

```
MergeRanges(R):
    sort R by start increasing
    out = empty list
    cur = R[0]
    for i in 1..len(R)-1:
        if R[i].start <= cur.end:
            cur.end = max(cur.end, R[i].end)
        else:
            append(out, cur)
            cur = R[i]
    append(out, cur)
    return out
```

Total time: $O(k \log k)$

Extra space: $O(k)$ output

Array Questions

Question 16: Common elements in 3 sorted arrays ($O(1)$ extra storage)

Given three arrays in non-decreasing order, find the elements that are common to all of them. Try $O(1)$ extra storage.

1213 - Intersection of Three Sorted Arrays: <https://leetcode.ca/2019-03-27-1213-Intersection-of-Three-Sorted-Arrays/>

<https://www.youtube.com/watch?v=zceoOrHSHNQ>

Solution 1 (Brute Force)

```
Common3_Brute(A, B, C):
    out = empty list
    for x in A:
        if x in B and x in C:
            append(out, x)
    return out
```

Total time: $O(n_1 \cdot (n_2 + n_3))$ (if linear membership)

Extra space: $O(1)$ extra + output

Solution 2 (Improved)

```
Common3(A, B, C):
    i = 0; j = 0; k = 0
    out = empty list
    while i < len(A) and j < len(B) and k < len(C):
        if A[i] == B[j] and B[j] == C[k]:
            append(out, A[i])
            i += 1; j += 1; k += 1
        else:
            mn = min(A[i], B[j], C[k])
            if A[i] == mn: i += 1
            if B[j] == mn: j += 1
            if C[k] == mn: k += 1
    return out
```

Total time: $O(n_1 + n_2 + n_3)$

Extra space: $O(1)$ extra + output

Array Questions

Question 17: Two sorted arrays identical except one has an extra element

Consider two sorted arrays that are identical, except one array has an extra element. Find that element and its index.

Index of an Extra Element: <https://www.geeksforgeeks.org/problems/index-of-an-extra-element/1>

<https://www.youtube.com/watch?v=0p-A4Qt9zAQ>

Solution 1 (Brute Force)

```
ExtraElement_Linear(A, B):
    # assume len(A) = len(B)+1; if not, swap
    if len(B) > len(A): swap(A, B)
    i = 0
    while i < len(B):
        if A[i] != B[i]:
            return (A[i], i)
        i += 1
    return (A[len(A)-1], len(A)-1)
```

Total time: $O(n)$

Extra space: $O(1)$

Solution 2 (Improved)

```
ExtraElement_Binary(A, B):
    if len(B) > len(A): swap(A, B)
    lo = 0; hi = len(B)    # search first mismatch position
    while lo < hi:
        mid = (lo + hi) // 2
        if A[mid] == B[mid]:
            lo = mid + 1
        else:
            hi = mid
    # lo is first mismatch index
    return (A[lo], lo)
```

Total time: $O(\log n)$

Extra space: $O(1)$

Array Questions

Question 18: Closest element in sorted array to x

Given a sorted array, find the closest element (in terms of absolute difference) to a given number.

35. Search Insert Position: <https://leetcode.com/problems/search-insert-position>

<https://www.youtube.com/watch?v=K-RYzDZkzCI>

Solution 1 (Brute Force)

```
ClosestSorted_Brute(A, x):
    best = A[0]
    bestDiff = abs(A[0] - x)
    for i in 1..len(A)-1:
        if abs(A[i] - x) < bestDiff:
            bestDiff = abs(A[i] - x)
            best = A[i]
    return best
```

Total time: $O(n)$

Extra space: $O(1)$

Solution 2 (Improved)

```
ClosestSorted(A, x):
    # binary search insertion point
    lo = 0; hi = len(A) # [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if A[mid] < x: lo = mid + 1
        else: hi = mid

    # lo is first index with A[lo] >= x
    if lo == 0: return A[0]
    if lo == len(A): return A[len(A)-1]
    if abs(A[lo] - x) < abs(A[lo-1] - x): return A[lo]
    return A[lo-1]
```

Total time: $O(\log n)$

Extra space: $O(1)$

Array Questions

Question 19: Find a local maximum in 1D array

Given an array, a local maximum is an element $A[i]$ such that is greater than or equal to the neighboring elements in the array. (There can be two neighboring elements, or just one if $i = 0$ or i is the last index.) Find a local maximum.

162. Find Peak Element: <https://leetcode.com/problems/find-peak-element>

<https://www.youtube.com/watch?v=kMzJy9es7Hc>

Solution 1 (Brute Force)

```
LocalMax1D_Brute(A):
    n = len(A)
    if n == 0: return NONE
    if n == 1: return 0
    if A[0] >= A[1]: return 0
    if A[n-1] >= A[n-2]: return n-1
    for i in 1..n-2:
        if A[i] >= A[i-1] and A[i] >= A[i+1]:
            return i
    return NONE
```

Total time: $O(n)$

Extra space: $O(1)$

Solution 2 (Improved)

```
LocalMax1D(A):
    lo = 0; hi = len(A) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if A[mid] < A[mid+1]:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

Total time: $O(\log n)$

Extra space: $O(1)$

Array Questions

Question 20: Find a local maximum in 2D grid (4-neighbor)

In a 2D array, a local maximum is defined as before, but the neighbors are both vertical and horizontal. So the neighbors of $A[i][j]$ are $A[i-1][j]$, $A[i+1][j]$, $A[i][j-1]$, $A[i][j+1]$ (whenever they exist). Find a local maximum.

1901. Find a Peak Element II: <https://leetcode.com/problems/find-a-peak-element-ii>

<https://www.youtube.com/watch?v=hXtNKTNmZs>

Solution 1 (Brute Force)

```
LocalMax2D_Brute(M):
    n = number of
    rows for i in
    0..n-1:
        for j in 0..n-1:
            up = M[i-1][j] if i>0 else -INF
            down = M[i+1][j] if i<n-1 else -
            INF left = M[i][j-1] if j>0
            else -INF right = M[i][j+1] if j<n-
            1 else -INF
            if M[i][j] >= up and M[i][j] >= down and M[i][j] >= left and M[i][j] >= right:
                return (i, j)
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
LocalMax2D(M):
    # column-based peak finding
    return LocalMax2D_Rec(M, 0, n-
    1)

LocalMax2D_Rec(M, cL, cR):
    midC = (cL + cR) // 2
    # find row r of maximum in column
    midC r = argmax over rows i of
    M[i][midC]

    leftVal = M[r][midC-1] if midC>cL else -
    INF rightVal = M[r][midC+1] if midC<cR else
    -INF

    if M[r][midC] >= leftVal and M[r][midC] >=
        rightVal: # also >= up/down because it is
        column max at r return (r, midC)
    if leftVal > M[r][midC]:
        return LocalMax2D_Rec(M, cL, midC-1)
    else:
        return LocalMax2D_Rec(M, midC+1, cR)
```

Total time: $O(n \log n)$

Extra space: $O(\log n)$ recursion

Array Questions

Question 21: Find if singly linked list has a loop (O(1) extra)

Find if a singly linked list has a loop. Use O(1) extra memory.

141. Linked List Cycle : <https://leetcode.com/problems/linked-list-cycle>

<https://www.youtube.com/watch?v=gBT7IFR3vc>

Solution 1 (Brute Force)

```
HasCycle_Brute(head):  
    # if you are allowed extra memory:  
    seen = empty set  
    cur = head  
    while cur != NULL:  
        if cur in seen: return true  
        add cur to seen  
        cur = cur.next  
    return false
```

Total time: O(n)

Extra space: O(n)

Solution 2 (Improved)

```
HasCycle(head):  
    slow = head  
    fast = head  
    while fast != NULL and fast.next != NULL:  
        slow = slow.next  
        fast = fast.next.next  
        if slow == fast:  
            return true  
    return false
```

Total time: O(n)

Extra space: O(1)

Array Questions

Question 22: Find first node on the loop (O(1) extra)

Going beyond the problem above, find the first node on the loop (assume there is a single loop). Use O(1) extra memory.

142. Linked List Cycle II: <https://leetcode.com/problems/linked-list-cycle-ii>

<https://www.youtube.com/watch?v=Yn5xgbK95Uw>

Solution 1 (Brute Force)

```
FirstCycleNode_Brute(head):
    seen = empty map node->index
    cur = head
    idx = 0
    while cur != NULL:
        if cur in seen:
            return cur
        seen[cur] = idx
        idx += 1
        cur = cur.next
    return NULL
```

Total time: O(n)

Extra space: O(n)

Solution 2 (Improved)

```
FirstCycleNode(head):
    # phase 1: detect meeting point
    slow = head
    fast = head
    while fast != NULL and fast.next != NULL:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            break
    if fast == NULL or fast.next == NULL:
        return NULL

    # phase 2: find entry
    p1 = head
    p2 = slow
    while p1 != p2:
        p1 = p1.next
        p2 = p2.next
    return p1
```

Total time: O(n)

Extra space: O(1)

Array Questions

Question 23: Stock market max profit (buy then later sell)

Consider the "stock market" problem. There is an input array A of prices, of (say) one unit of stock. You need to find out the maximum profit that can be made, assuming you buy (one unit of) stock on a day and sell it on a subsequent day.

121. Best Time to Buy and Sell Stock: <https://leetcode.com/problems/best-time-to-buy-and-sell-stock>

<https://www.youtube.com/watch?v=1pkOgXD63yU>

Solution 1 (Brute Force)

```
MaxProfit_Brute(A):
    best = 0
    for i in 0..len(A)-1:
        for j in i+1..len(A)-1:
            best = max(best, A[j] - A[i])
    return best
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MaxProfit(A):
    minSoFar = +INF
    best = 0
    for x in A:
        best = max(best, x - minSoFar)
        minSoFar = min(minSoFar, x)
    return best
```

Total time: $O(n)$

Extra space: $O(1)$

Array Questions

Question 24: Best selling day for every day i

The stock market problem again. Except, you need to determine the best selling day, for every single day. That is, for each i , find $j > i$ that maximizes $A[j] - A[i]$.

122. Best Time to Buy and Sell Stock II: <https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii>

<https://www.youtube.com/watch?v=3SJ3pUkPQMc>

Solution 1 (Brute Force)

```
BestSellDay_Brute(A):
    n = len(A)
    ans = array size n filled -1
    for i in 0..n-1:
        bestProfit = 0
        bestJ = -1
        for j in i+1..n-1:
            if A[j] - A[i] > bestProfit:
                bestProfit = A[j] - A[i]
                bestJ = j
        ans[i] = bestJ
    return ans
```

Total time: $O(n^2)$

Extra space: $O(n)$ output

Solution 2 (Improved)

```
BestSellDay(A):
    n = len(A)
    ans = array size n filled -1
    maxPrice = A[n-1]
    maxIdx = n-1
    ans[n-1] = -1

    for i in n-2 down to 0:
        if maxPrice > A[i]:
            ans[i] = maxIdx
        else:
            ans[i] = -1
            maxPrice = A[i]
            maxIdx = i
    return ans
```

Total time: $O(n)$

Extra space: $O(n)$ output

Array Questions

Question 25: Compute permutation π of final sorted positions

Every array A induces a permutation π where $\pi(i)$ is the final location of the element $A[i]$ in the sorted version of A . Thus, element $A[i]$ ends up at index $\pi(i)$ after sorting. Given A , determine this permutation π .

1331. Rank Transform of an Array: <https://leetcode.com/problems/rank-transform-of-an-array>

<https://www.youtube.com/watch?v=f1nZp5Mr3XE>

Solution 1 (Brute Force)

```
Permutation_Brute(A):
    B = copy of A
    sort(B)
    pi = array size len(A)
    for i in 0..len(A)-1:
        # find A[i] in B by linear search
        for j in 0..len(B)-1:
            if B[j] == A[i]:
                pi[i] = j
                break
    return pi
```

Total time: $O(n^2) + O(n \log n)$

Extra space: $O(n)$ (copy)

Solution 2 (Improved)

```
Permutation(A):
    pairs = empty list
    for i in 0..len(A)-1:
        append(pairs, (A[i], i))    # (value, originalIndex)
    sort(pairs)                    # by value
    pi = array size len(A)
    for pos in 0..len(pairs)-1:
        (_, orig) = pairs[pos]
        pi[orig] = pos
    return pi
```

Total time: $O(n \log n)$

Extra space: $O(n)$

Array Questions

Question 26: MergeSort variant 1/3–2/3 split

Consider a version of MergeSort that does a 1/3-2/3 re split, instead of in halves. Meaning, the “left” subarray will be the first third of elements, and the “right” array is formed by the remaining elements. What is the running time of this version of MergeSort?

148. Sort List: <https://leetcode.com/problems/sort-list>

<https://www.youtube.com/watch?v=TGveA1oFhrc>

Solution 1 (Brute Force)

```
RunningTimeArgument_Brute():  
    # Use recursion tree idea: each level does O(n) merging work.  
    # Depth is O(log n) because subproblems shrink geometrically.  
    return "O(n log n)"
```

Total time: $O(1)$ to state

Extra space: $O(1)$

Solution 2 (Improved)

```
RunningTimeArgument_MoreFormal():  
    # Recurrence:  $T(n) = T(n/3) + T(2n/3) + O(n)$   
    # Total work per level is  $O(n)$ ; depth is  $O(\log n)$ .  
    return " $T(n) = O(n \log n)$ "
```

Total time: $O(1)$ to state

Extra space: $O(1)$

Array Questions

Question 27: Sorted distinct A, determine if exists i with $A[i]=i$

Given a sorted array A of distinct integers, determine if there is an index i such that $A[i] = i$.

1064 - Fixed Point: <https://leetcode.ca/2018-10-29-1064-Fixed-Point/>

https://www.youtube.com/watch?v=wYTIWnq29_s

<https://www.youtube.com/watch?v=eBdu51ZfFkU>

Solution 1 (Brute Force)

```
HasFixedPoint_Brute(A):
    for i in 0..len(A)-1:
        if A[i] == i: return true
    return false
```

Total time: $O(n)$

Extra space: $O(1)$

Solution 2 (Improved)

```
HasFixedPoint(A):
    lo = 0; hi = len(A)-1
    while lo <= hi:
        mid = (lo + hi) // 2
        if A[mid] == mid: return true
        if A[mid] < mid:
            lo = mid + 1
        else:
            hi = mid - 1
    return false
```

Total time: $O(\log n)$

Extra space: $O(1)$

Array Questions

Question 28: Very large array: first n entries positive, rest are 0; find n

There is an extremely large array A such that the first n entries are positive integers, and all remaining entries are 0. Find n .

278. First Bad Version: <https://leetcode.com/problems/first-bad-version>

<https://www.youtube.com/watch?v=vnfGi-ucwTE>

Solution 1 (Brute Force)

```
FindN_Brute(A):  
    i = 0  
    while A[i] != 0:  
        i += 1  
    return i
```

Total time: $O(n)$

Extra space: $O(1)$

Solution 2 (Improved)

```
FindN(A):  
    # exponential search to find upper bound  
    hi = 1  
    while A[hi] != 0:  
        hi *= 2  
    lo = hi // 2  
  
    # binary search first zero  
    while lo < hi:  
        mid = (lo + hi) // 2  
        if A[mid] != 0:  
            lo = mid + 1  
        else:  
            hi = mid  
    return lo
```

Total time: $O(\log n)$

Extra space: $O(1)$

Array Questions

Question 29: Search x in $n \times n$ matrix sorted by rows and columns

Matrix has increasing rows and columns. Determine if x is in the matrix.

240. Search a 2D Matrix II: <https://leetcode.com/problems/search-a-2d-matrix-ii>

<https://www.youtube.com/watch?v=Sz-ljAuPzqw>

Solution 1 (Brute Force)

```
MatrixSearch_Brute(M, x):
    n = number of rows
    for i in 0..n-1:
        for j in 0..n-1:
            if M[i][j] == x: return true
    return false
```

Total time: $O(n^2)$

Extra space: $O(1)$

Solution 2 (Improved)

```
MatrixSearch(M, x):
    n = number of rows
    i = 0
    j = n - 1    # start top-right
    while i < n and j >= 0:
        if M[i][j] == x: return true
        if M[i][j] > x:
            j -= 1
        else:
            i += 1
    return false
```

Total time: $O(n)$

Extra space: $O(1)$

Data Structure Questions

Question 1: Stack with push/pop/findMin all O(1)

Describe a stack based data structure that supports push, pop, and find- min in O(1) operations.

155. Min Stack: <https://leetcode.com/problems/min-stack/>

<https://www.youtube.com/watch?v=gkLI7nAwDPo>

Solution 1 (Brute Force)

```
MinStack_Brute():
    S = empty stack
    push(x): S.push(x)
    pop(): return S.pop()
    findMin():
        # scan entire stack (conceptually)
        m = +INF
        for each item in S:
            m = min(m, item)
        return m
```

Total time: push/pop O(1), findMin O(n)

Extra space: O(1) extra

Solution 2 (Improved)

```
MinStack():
    S = empty stack
    M = empty stack # mins

    push(x):
        S.push(x)
        if M.empty or x <= M.top:
            M.push(x)

    pop():
        x = S.pop()
        if x == M.top:
            M.pop()
        return x

    findMin():
        return M.top
```

Total time: All ops O(1)

Extra space: O(n) (second stack)

Data Structure Questions

Question 2: Reconstruct BST from preorder traversal

Given the pre-order traversal of a binary search tree, reconstruct the tree.

1008. Construct Binary Search Tree from Preorder Traversal: <https://leetcode.com/problems/construct-binary-search-tree-from-preorder-traversal>

<https://www.youtube.com/watch?v=9sw8RRsBw6s>

Solution 1 (Brute Force)

```
BuildBST_InsertAll(pre):
    root = NULL
    for x in pre:
        root = BST_Insert(root, x)
    return root
```

Total time: $O(nH)$ (worst $O(n^2)$)

Extra space: $O(1)$ extra (ignoring recursion)

Solution 2 (Improved)

```
BuildBST_FromPreorder(pre):
    idx = 0
    return Build(pre, -INF, +INF, idx)

Build(pre, low, high, idx by reference):
    if idx == len(pre): return NULL
    x = pre[idx]
    if x < low or x > high: return NULL

    idx += 1
    node = new Node(x)
    node.left = Build(pre, low, x, idx)
    node.right = Build(pre, x, high, idx)
    return node
```

Total time: $O(n)$

Extra space: $O(H)$ recursion

Data Structure Questions

Question 3: Queue using only stacks; total for k ops

Design a queue using only stacks. It must support the enqueue and de-queue operations. If you start with an empty queue, end up enqueueing and dequeuing k elements, what is the total running time using your data structure?

232. Implement Queue using Stacks: <https://leetcode.com/problems/implement-queue-using-stacks>

<https://www.youtube.com/watch?v=eanwa3ht3YQ>

Solution 1 (Brute Force)

```
QueueWithStacks_Slow():
    S = empty stack

    enqueue(x):
        T = empty stack
        while S not empty: T.push(S.pop())
        S.push(x)
        while T not empty: S.push(T.pop())

    dequeue():
        return S.pop()
```

Total time: enqueue $O(n)$, dequeue $O(1)$, total up to $O(k^2)$

Extra space: $O(k)$ (temporary stack)

Solution 2 (Improved)

```
QueueWithStacks():
    IN = empty stack
    OUT = empty stack

    enqueue(x):
        IN.push(x)

    dequeue():
        if OUT.empty:
            while IN not empty:
                OUT.push(IN.pop())
        return OUT.pop()
```

Total time: Amortized $O(1)$ per op; total $O(k)$

Extra space: $O(k)$

Data Structure Questions

Question 4: Login/logout storage; query counts in time range

Given login/logout time of all users for a particular website in the form: (userId, login time, logout time). Store this data, to query the total number of users who logged in and logged out in a given time range.

2080. Range Frequency Queries: <https://leetcode.com/problems/range-frequency-queries>

<https://www.youtube.com/watch?v=faj3fMRQo68>

Solution 1 (Brute Force)

```
LoginLogout_Brute(records):
    logins = list of all login times
    logouts = list of all logout times

    query(a, b):
        c1 = 0; c2 = 0
        for t in logins:
            if a <= t <= b: c1 += 1
        for t in logouts:
            if a <= t <= b: c2 += 1
        return (c1, c2)
```

Total time: query $O(N)$

Extra space: $O(N)$

Solution 2 (Improved)

```
LoginLogout():
    logins = sorted list of login times
    logouts = sorted list of logout times

    build(records):
        fill logins and logouts
        sort(logins); sort(logouts)

    countInRange(sortedTimes, a, b):
        # returns number of t with a <= t <= b
        left = lower_bound(sortedTimes, a)
        right = upper_bound(sortedTimes, b)
        return right - left

    query(a, b):
        return (countInRange(logins, a, b), countInRange(logouts, a, b))
```

Total time: build $O(N \log N)$; query $O(\log N)$

Extra space: $O(N)$

Data Structure Questions

Question 5: Random song player with insert/delete/playRandom

I am designing a media player that should store songs and play them in random order. I need to be able to insert, delete, and play a random song. What data structure would you use?

380. Insert Delete GetRandom O(1): <https://leetcode.com/problems/insert-delete-getrandom-o1>

<https://www.youtube.com/watch?v=j4KwhBziOpg>

Solution 1 (Brute Force)

```
RandomSongs_Brute():
    S = list of songs

    insert(song):
        append(S, song)

    delete(song):
        find song in S by linear scan and remove (shift)

    playRandom():
        i = randomInt(0, len(S)-1)
        play(S[i])
```

Total time: insert $O(1)$, delete $O(n)$, random $O(1)$

Extra space: $O(n)$

Solution 2 (Improved)

```
RandomSongs():
    A = dynamic array of songs
    H = hash map song -> index in A

    insert(song):
        if song in H: return
        H[song] = len(A)
        append(A, song)

    delete(song):
        if song not in H: return
        i = H[song]
        last = A[len(A)-1]
        A[i] = last
        H[last] = i
        remove last element from A
        delete H[song]

    playRandom():
        i = randomInt(0, len(A)-1)
        play(A[i])
```

Total time: All ops average $O(1)$

Extra space: $O(n)$

Data Structure Questions

Question 6: Unique path between two BST nodes x and y

Given two pointers to nodes x and y in a binary search tree, find the unique path between them. What is the time complexity of your algorithm?

235. Lowest Common Ancestor of a Binary Search Tree: <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree>

<https://www.youtube.com/watch?v=gs2LMfuOR9k>

Solution 1 (Brute Force)

```
PathBST_Brute(root, x, y):
    # find root-to-x path by DFS, and root-to-y path by DFS
    px = RootToNodePath(root, x)
    py = RootToNodePath(root, y)
    # remove common prefix to find LCA, then combine
    return Combine(px, py)
```

Total time: $O(n)$

Extra space: $O(H)$ recursion + path storage

Solution 2 (Improved)

```
PathBST(root, x, y):
    lca = LCA_BST(root, x.key, y.key)
    p1 = PathFrom(lca, x.key)    # follow BST decisions
    p2 = PathFrom(lca, y.key)

    # unique path x->y is reverse(p1) + p2[1..]
    return reverse(p1) concatenated with p2 without first node

LCA_BST(node, a, b):
    while node != NULL:
        if a < node.key and b < node.key: node = node.left
        else if a > node.key and b > node.key: node = node.right
        else: return node

PathFrom(node, target):
    path = empty list
    cur = node
    while cur != NULL:
        append(path, cur)
        if target == cur.key: break
        if target < cur.key: cur = cur.left
        else: cur = cur.right
    return path
```

Total time: $O(H)$

Extra space: $O(H)$ (path output)

Data Structure Questions

Question 7: Stream of words, report top 10 most repeated

You have a stream of words. Design a data structure that at any point, tells you the 10 most repeated words.

347. Top K Frequent Elements: <https://leetcode.com/problems/top-k-frequent-elements>

<https://www.youtube.com/watch?v=YPTqKlgVk-k>

and

692. Top K Frequent Words: <https://leetcode.com/problems/top-k-frequent-words>

<https://www.youtube.com/watch?v=x9MGKyY7GqU>

Solution 1 (Brute Force)

```
Top10Words_Brute():
    words = list

    insert(w):
        append(words, w)

    top10():
        # compute frequencies by nested scans
        freqPairs = empty list
        for each w in words:
            count = 0
            for each u in words:
                if u == w: count += 1
            update freqPairs for w with count
        sort freqPairs by count descending
        return first 10 words
```

Total time: insert $O(1)$, top10 $O(n^2 + n \log n)$

Extra space: $O(n)$

Solution 2 (Improved)

```
Top10Words():
    count = hash map word -> frequency
    heap = min-heap of at most 10 items (freq, word)

    insert(w):
        count[w] += 1

    top10():
        heap.clear()
        for each (w, f) in count:
            if heap.size < 10:
                heap.push((f, w))
            else if f > heap.top.freq:
                heap.popMin()
                heap.push((f, w))
        return heap items sorted descending by freq
```

Total time: insert avg $O(1)$, top10 $O(U \log 10)$ where $U = \# \text{unique}$

Extra space: $O(U) + O(10)$

Data Structure Questions

Question 8: Stream of integers; maintain median; what about kth smallest

You have a stream of integers. Maintain the value of the median. What about the kth smallest element?

703. Kth Largest Element in a Stream <https://leetcode.com/problems/kth-largest-element-in-a-stream>

<https://www.youtube.com/watch?v=hOjcdrgMoQ8>

Solution 1 (Brute Force)

```
StreamOrderStats_Brute():
    A = empty list
    insert(x):
        append(A, x)
        sort(A)
    median():
        n = len(A)
        if n%2==1: return A[n//2]
        return (A[n//2-1] + A[n//2]) / 2
    kth(k):
        return A[k-1]
```

Total time: insert $O(n \log n)$, queries $O(1)$

Extra space: $O(n)$

Solution 2 (Improved)

```
MedianTwoHeaps():
    maxHeap = empty max-heap
    minHeap = empty min-heap

    insert(x):
        if maxHeap.empty() or x <= maxHeap.top(): maxHeap.push(x)
        else: minHeap.push(x)
        if maxHeap.size > minHeap.size + 1:
            minHeap.push(maxHeap.popTop())
        if minHeap.size > maxHeap.size:
            maxHeap.push(minHeap.popTop())

    median():
        if maxHeap.size == minHeap.size:
            return (maxHeap.top + minHeap.top) / 2
        return maxHeap.top

KthSmallest_Note():
    # If your course covered augmented BSTs (order statistic tree),
    # store subtree sizes to support insert and select(k) in  $O(\log n)$ .
    # Otherwise, a practical approach is to keep a balanced BST/multiset and walk k steps.
```

Total time: median insert $O(\log n)$, median $O(1)$; kth with OST: $O(\log n)$

Extra space: $O(n)$

Data Structure Questions

Question 9: Count violating ancestor-descendant pairs in (supposed) BST

A pair of nodes x, y in a (supposed) binary search tree violate the BST property if x is an ancestor of y , and the corresponding values are "out of order". Given a BST, find the number of pairs that violate the BST property.

98. Validate Binary Search Tree: <https://leetcode.com/problems/validate-binary-search-tree/>

Solution 1 (Brute Force)

```
CountViolations_Brute(root):
    nodes = list of all nodes (e.g.,
    preorder)
    violations = 0
    for each node x in nodes:
        for each descendant y of x:
            if (y is in left subtree of x and y.key > x.key)
            or (y is in right subtree of x and y.key <
            x.key):
                violations
                += 1
    return violations
```

Total time: $O(n^2)$

Extra space: $O(n)$ (node list)

Solution 2 (Improved)

```
CountViolations_Improved(root):
    # A common improvement uses recursion carrying valid (min,max) bounds
    and # additional counting structure; exact optimal depends on course
    coverage. # Safe approach for this test: present the brute-force
    clearly.
    return CountViolations_Brute(root)
```

Total time: $O(n^2)$

Extra space: $O(n)$

Data Structure Questions

Question 10: Valid parentheses/brackets string

Given a string of parentheses/brackets, check if it is valid (meaning that every opening can be matched to a closing). Thus, $((\{(\}\}))()$ is valid, but $()(\{)$ is not.

20. Valid Parentheses: <https://leetcode.com/problems/valid-parentheses>

<https://www.youtube.com/watch?v=WTzjTskDFMg>

Solution 1 (Brute Force)

```
IsValid_Brute(s):
    # repeatedly delete adjacent matching pairs until no change
    changed = True
    while changed:
        changed = False
        i = 0
        while i < len(s)-1:
            if s[i..i+1] is one of '()', '[]', '{}':
                delete s[i..i+1]
                changed = True
                break
            i += 1
    return len(s) == 0
```

Total time: $O(n^2)$ worst

Extra space: $O(1)$ extra (in-place string edits)

Solution 2 (Improved)

```
IsValid(s):
    st = empty stack
    for ch in s:
        if ch is opening bracket:
            st.push(ch)
        else:
            if st.empty: return False
            top = st.pop()
            if not matches(top, ch): return False
    return st.empty

matches(open, close):
    return (open=='(' and close==')') or (open=='[' and close==']') or (open=='{' and close=='}')
```

Total time: $O(n)$

Extra space: $O(n)$

Data Structure Questions

Question 11: Sort a k-sorted array in $O(n \log k)$

An array is k-sorted, if all elements are misplaced by at most k positions (with respect to the true sorted order). Sort a k-sorted array in time $O(n \log k)$.

215. Kth Largest Element in an Array: <https://leetcode.com/problems/kth-largest-element-in-an-array>

<https://www.youtube.com/watch?v=XEmy13g1Qxc>

Solution 1 (Brute Force)

```
SortKSorted_Brute(A):  
    sort(A)  
    return A
```

Total time: $O(n \log n)$

Extra space: $O(1)$ extra (depends on sort)

Solution 2 (Improved)

```
SortKSorted(A, k):  
    heap = empty min-heap  
    out = empty list  
  
    for i in 0..min(k, len(A)-1):  
        heap.push(A[i])  
  
    for i in k+1..len(A)-1:  
        append(out, heap.popMin())  
        heap.push(A[i])  
  
    while heap not empty:  
        append(out, heap.popMin())  
  
    copy out back into A (or return out)
```

Total time: $O(n \log k)$

Extra space: $O(k)$ heap + $O(n)$ output

Data Structure Questions

Question 12: Convert max heap to min heap

Convert a max heap to a min heap.

Convert Min Heap to Max Heap: <https://www.geeksforgeeks.org/dsa/convert-min-heap-to-max-heap>

<https://www.youtube.com/watch?v=t0Cq6tVNRBA>

Solution 1 (Brute Force)

```
MaxToMin_Brute(A):  
    # remove all elements then reinsert into a new min-heap  
    B = empty min-heap  
    while A not empty:  
        x = extractMax(A)  
        B.insert(x)  
    return B
```

Total time: $O(n \log n)$

Extra space: $O(n)$

Solution 2 (Improved)

```
MaxToMin(A):  
    # bottom-up heapify for min-heap  
    for i in (len(A)//2 - 1) down to 0:  
        MinHeapify(A, i)  
  
MinHeapify(A, i):  
    while true:  
        l = 2*i + 1  
        r = 2*i + 2  
        smallest = i  
        if l < len(A) and A[l] < A[smallest]: smallest = l  
        if r < len(A) and A[r] < A[smallest]: smallest = r  
        if smallest == i: break  
        swap(A[i], A[smallest])  
        i = smallest
```

Total time: $O(n)$

Extra space: $O(1)$

Data Structure Questions

Question 13: Smallest range intersecting all k sorted arrays

Consider an input of k sorted arrays of integers, each of size n. A range [a, b] intersects an array, if it contains an element of the array. The size of this range is |b - a|. Find the smallest range intersecting all arrays.

632. Smallest Range Covering Elements from K Lists: <https://leetcode.com/problems/smallest-range-covering-elements-from-k-lists>

https://www.youtube.com/watch?v=L_0aPFMgGpU

Solution 1 (Brute Force)

```
SmallestRange_Brute(Arrays):
    # check all pairs (a,b) from all values and test coverage
    vals = all elements from all arrays
    sort(vals)
    best = (vals[0], vals[-1])
    for i in 0..len(vals)-1:
        for j in i..len(vals)-1:
            if CoversAll(Arrays, vals[i], vals[j]):
                if (vals[j]-vals[i]) < (best.b - best.a):
                    best = (vals[i], vals[j])
    return best

CoversAll(Arrays, a, b):
    for each arr in Arrays:
        if no element in arr within [a,b]: return false
    return true
```

Total time: Very large ($\geq O(N^2 * k \log n)$)

Extra space: $O(N)$

Solution 2 (Improved)

```
SmallestRange(Arrays):
    heap = min-heap of (value, arrayId, index)
    curMax = -INF
    for t in 0..k-1:
        v = Arrays[t][0]
        heap.push((v, t, 0))
        curMax = max(curMax, v)

    bestA = heap.top.value
    bestB = curMax

    while true:
        (v, t, i) = heap.popMin()
        if curMax - v < bestB - bestA:
            bestA = v; bestB = curMax

        if i + 1 == len(Arrays[t]):
            break
        nxt = Arrays[t][i+1]
        heap.push((nxt, t, i+1))
        curMax = max(curMax, nxt)

    return (bestA, bestB)
```

Total time: $O(N \log k)$ where N total elements

Extra space: $O(k)$ heap

Data Structure Questions

Question 14: Augment BST to track node heights; update on insert/delete

idea: have each node hold an int for height, this will add to the constant multiple of extra space so still $O(n)$ total space. inserts into a regular binary search tree are always leaves so when inserting set height to 1. for delete the only heights that change are any "above" the one that gets deleted. using a recursive delete function decrement a node's height by one after all recursive calls so that it will happen when the code moves back 'up' the stack

701. Insert into a Binary Search Tree: <https://leetcode.com/problems/insert-into-a-binary-search-tree>

<https://www.youtube.com/watch?v=Cpg8f79luEA>

and

450. Delete Node in a BST: <https://leetcode.com/problems/delete-node-in-a-bst>

<https://www.youtube.com/watch?v=LFzAoJJt92M>

Solution 1 (Brute Force)

```
UpdateHeights_Brute(root):
    # recompute heights of entire tree after each operation
    return Recompute(root)

Recompute(node):
    if node == NULL: return 0
    lh = Recompute(node.left)
    rh = Recompute(node.right)
    node.height = 1 + max(lh, rh)
    return node.height
```

Total time: $O(n)$ per update

Extra space: $O(H)$ recursion

Solution 2 (Improved)

```
BST_Insert_WithHeights(root, x):
    root = BST_Insert(root, x)    # standard BST insert returns inserted node path known
    via recu # update heights while unwinding recursion:
    # node.height = 1 + max(height(node.left), height(node.right))
    return root

BST_Delete_WithHeights(root, x):
    root = BST_Delete(root, x)
    # similarly update heights while unwinding recursion
    return root

Height(node):
    return 0 if node==NULL else node.height
```

Total time: $O(H)$ per insert/delete

Extra space: $O(H)$ recursion

Data Structure Questions

Question 15: BST less-than delete (delete all keys $\leq x$)

Consider a BST. We want an extra operation "less-than delete": given x , this operation should delete all keys at most x . Design an efficient algorithm for this problem. You are allowed to augment your tree with extra $O(1)$ information per node.

669. Trim a Binary Search Tree: <https://leetcode.com/problems/trim-a-binary-search-tree>

<https://www.youtube.com/watch?v=jwt5mTjEXGc>

Solution 1 (Brute Force)

```
LessThanDelete_Brute(root, x):
    # inorder traversal, delete each key  $\leq x$  individually
    keys = InorderKeys(root)
    for k in keys:
        if k  $\leq$  x:
            root = BST_Delete(root, k)
    return root
```

Total time: $O(m \cdot H)$ where $m = \# \text{deleted}$

Extra space: $O(n)$ (keys list)

Solution 2 (Improved)

```
LessThanDelete(root, x):
    if root == NULL: return NULL
    if root.key  $\leq$  x:
        # delete root and entire left subtree; keep processing right subtree
        return LessThanDelete(root.right, x)
    else:
        root.left = LessThanDelete(root.left, x)
        return root
```

Total time: $O(\text{nodes visited})$ (often $O(H + \text{deleted})$)

Extra space: $O(H)$ recursion